



Computer Vision and Image Processing

Lecture 14

Edge Detection

- An **edge in an image** is a boundary or contour where the **image intensity (brightness or colour) changes sharply** over a short distance.
- Edges are critical features in computer vision and image processing because they represent the important structural information of a scene, such as the **boundaries of objects**, changes in surface orientation, or variations in lighting.
- Edge detection identifies **rapid intensity changes** in an image, which typically correspond to **object boundaries, texture changes, or depth discontinuities**.

Edge Types

Edge Type	Description (Intensity Change)	Derivative Tool
Step Edge	An abrupt change from one intensity level to another. (Ideal)	First Derivative has a sharp peak (maximum magnitude). Second Derivative has a zero-crossing (changes sign) at the edge center.
Ramp Edge	A gradual change in intensity over a short distance. (More realistic)	First Derivative has a peak over the ramp's width. Second Derivative has a zero-crossing at the centre of the ramp.
Roof Edge	A peak or ridge in intensity, often corresponding to lines thinner than the filter size.	First Derivative has two peaks of opposite sign. Second Derivative has a strong minimum (for a positive roof/ridge) at the centre.

Edge Profiles

This is the **plot of intensity values** along a cross-section of the image.

- The **shape** of the plot indicates the type of edge.
- Used to understand how smooth or abrupt the change is.

Example:

- Step edge → sharp vertical jump.
- Ramp edge → slanted line.
- Roof edge → triangular peak.

Edge Operators

These are filters (kernels) that detect changes in intensity.

Main categories:

- **Gradient-based operators** → detect edges by finding *first derivative*.
- **Laplacian-based operators** → detect edges by finding *second derivative*.

Sobel Operator (First-Order Derivative)

The Sobel operator is a gradient-based operator that approximates the first derivative. It uses two 3*3 convolution kernels: **G_x** for the horizontal derivative (detects vertical edges) and **G_y** for the vertical derivative (detects horizontal edges). The center row/column is weighted by 2 for increased smoothing and noise reduction.

Horizontal gradient (G_x):

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Vertical gradient (G_y):

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Sobel Operator (First-Order Derivative)

Magnitude:

$$G = \sqrt{G_x^2 + G_y^2}$$

Direction:

$$\theta = \tan^{-1} \left(\frac{G_y}{G_x} \right)$$

Edge angle:

$$\theta_{edge} = \theta + 90^\circ$$

Sobel Operator (First-Order Derivative)

$$I = \begin{bmatrix} 10 & 10 & 80 \\ 10 & 10 & 80 \\ 10 & 10 & 80 \end{bmatrix}$$

Laplacian (Second Derivative Operator)

The Laplacian detects edges by checking **zero-crossings** of second derivatives.

Common kernel:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Or 8-connected:

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Laplacian (Second Derivative Operator)

Characteristics

- **Positive Laplacian value:** centre pixel is a local intensity **peak** relative to neighbours (second derivative positive).
- **Negative Laplacian value:** centre is a local **valley** (second derivative negative).
- **Zero or near 0:** flat region or balanced neighbourhood.
- **Zero-crossings between adjacent pixels:** indicate possible edges (where Laplacian changes sign).

Because Laplacian measures the **second derivative**, it responds strongly where the first derivative changes quickly — i.e., at edges. But the second derivative is **very sensitive to noise**, so in practice you usually:

1. Smooth the image with a Gaussian (to reduce noise), then
2. Apply the Laplacian (the combined operator is called **LoG — Laplacian of Gaussian**), then
3. Detect zero-crossings (possibly with a threshold on magnitude).

Laplacian (Second Derivative Operator)

$$I = \begin{pmatrix} 10 & 10 & 10 & \mathbf{200} & 200 \\ 10 & 10 & 10 & \mathbf{200} & 200 \\ 10 & 10 & 10 & \mathbf{200} & 200 \\ 10 & 10 & 10 & \mathbf{200} & 200 \\ 10 & 10 & 10 & \mathbf{200} & 200 \end{pmatrix}$$

Image Smoothing

- Image smoothing removes high-frequency noise, producing a soft/blurred image. It can be done in **spatial domain** or **frequency domain**.
- Image Smoothing can be applied in both **Spatial** and **Frequency** Domain.

Image Smoothing (Spatial Domain)

- Spatial Domain Smoothing Filters:
 1. Averaging (Mean) Filter
 2. Weighted Averaging Filter
 3. Gaussian Smoothing Filter
 4. Median Filter
 5. Bilateral Filter (*edge-preserving*)

Averaging (Mean) Filter

- The averaging filter replaces each pixel with the **average** of its neighbouring pixels.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

For a pixel (x, y) :

$$g(x, y) = \frac{1}{mn} \sum_{i=-a}^a \sum_{j=-b}^b f(x+i, y+j)$$

Averaging (Mean) Filter

Property	Behaviour
Blurring strength	High (strong smoothing)
Noise removal	Good for random noise
Edge preservation	Poor (edges softened)
Computational cost	Very low

Use Cases

- Removing random noise
- Pre-processing before edge detection
- Reducing details in an image

Causes **strong blurring** and loses fine details.

Weighted Averaging Filter

- Assigns **higher weights** to central pixels and **lower weights** to neighbours.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

This gives more importance to the center pixel → smoother and less blurry than simple averaging.

Weighted Averaging Filter

Property

Behaviour

Blurring

Moderate

Noise removal

Better than simple averaging

Edge preservation

Better than uniform averaging

Smoothness

Higher

Use Cases

- Natural image smoothing
- Removing mild noise
- Pre-processing for computer vision tasks

Gaussian Smoothing Filter

- Uses a kernel based on the **2D Gaussian distribution**:

$$G(x, y) = e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Example (Gaussian kernel, $\sigma = 1$):

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

(Same pattern as weighted average but mathematically derived.)

Gaussian Smoothing Filter

Property	Behaviour
Blurring	Moderate
Ringing artifacts	None
Edge preservation	Good
Noise reduction	Excellent
Smoothness	Very natural

Use Cases

- Natural image denoising
- Removing Gaussian noise
- Pre-processing for:
 - Face detection
 - Object detection
 - Medical imaging
- Suitable where visual quality matters

Gaussian filter produces the **most visually pleasant smoothing** in spatial domain.

Median Filter (Nonlinear)

- Replaces each pixel with the **median** of its neighbours values.

Neighborhood:

10 200 15

18 20 22

21 16 19

Sorted:

10, 15, 16, 18, 19, 20, 21, 22, 200

Median = 19

Median Filter (Nonlinear)

Property	Behaviour
Blurring	Very low
Edge preservation	Excellent
Type	Non-linear

Use Cases

- Salt-and-pepper noise removal
- Preserving sharp edges (e.g., medical images)
- Removing outliers in sensor data

Median filter is the **best filter** for salt-and-pepper noise.

Bilateral Filter (Advanced Edge-preserving Smoothing)

- Performs smoothing based on:

1. Spatial closeness

2. Pixel intensity similarity

$$g(x, y) = \frac{1}{W} \sum_i \sum_j f(i, j) \cdot e^{-\frac{(x-i)^2 + (y-j)^2}{2\sigma_s^2}} \cdot e^{-\frac{(f(x, y) - f(i, j))^2}{2\sigma_r^2}}$$

Bilateral Filter (Advanced Edge-preserving Smoothing)

Property	Behaviour
Edge preservation	Best among smoothing filters
Blurring	Very low
Noise removal	High
Complexity	High (slow)

Use Cases

- Face smoothing
- Image beautification
- Removing noise while preserving edges
- Depth maps and 3D images

Often used in **computer photography** (e.g., skin smoothing in smartphone cameras).

Comparison Summary (Spatial Domain Smoothing)

Filter	Type	Edge Preservation	Blurring	Best For
Mean (Averaging)	Linear	Poor	Strong	Basic noise reduction
Weighted Mean	Linear	Medium	Moderate	Natural smoothing
Gaussian	Linear	Good	Moderate	High-quality smoothing
Median	Non-linear	Excellent	Low	Salt-and-pepper noise
Bilateral	Non-linear	Excellent	Very low	Edge-preserving denoising

Comparison Summary (Spatial Domain Smoothing)

Which Spatial Filter Should You Use?

- For **random noise reduction** → **Averaging / Gaussian**
- For **natural smooth image** → **Gaussian**
- For **salt-and-pepper noise** → **Median**
- For **smoothing without losing edges** → **Bilateral**
- For **face/skin smoothing** → **Bilateral Filter**

Image Smoothing (Frequency Domain)

- The most common frequency-domain smoothing filters are:
 - 1. Ideal Low-Pass Filter (ILPF)**
 - 2. Butterworth Low-Pass Filter (BLPF)**
 - 3. Gaussian Low-Pass Filter (GLPF)**
- Each filter reduces high-frequency components differently — this affects image quality, ringing effects, and smoothness.

Ideal Low-Pass Filter (ILPF)

- An ideal LPF passes **all frequencies inside a radius D_0** and completely removes all frequencies outside it.

Mathematically:

$$H(u, v) = \begin{cases} 1, & D(u, v) \leq D_0 \\ 0, & D(u, v) > D_0 \end{cases}$$

Where

$$D(u, v) = \sqrt{(u - M/2)^2 + (v - N/2)^2}$$

Ideal Low-Pass Filter (ILPF)

Property

Cutoff shape

Passband

Stopband

Transition band

Artifacts

Smoothness

Complexity

Behaviour

Perfect circle (sharp boundary)

Frequencies inside D_0 fully preserved

Frequencies outside D_0 completely removed

None ($0 \rightarrow 1$ abruptly)

Strong **ringing** (Gibbs effect) in spatial domain

Poor (because of sharp cutoff)

Very simple

Ideal Low-Pass Filter (ILPF)

Applications

- Synthetic images
- Removing small details
- Educational purposes
- Demonstrating frequency cutoffs

Why reason

- No noise in edges, acceptable ringing
- Strong high-frequency removal
- Easy to visualize and compute
- Clear, abrupt frequency blocking

Not good for natural images, because the ringing artifacts distort edges.

Butterworth Low-Pass Filter (BLPF)

- A smoother version of ILPF. Instead of abrupt cut-offs, Butterworth decreases **gradually**.

$$H(u, v) = \frac{1}{1 + \left(\frac{D(u, v)}{D_0} \right)^{2n}}$$

Where n is the **order**.

- Higher $n \rightarrow$ more similar to Ideal LPF
- Lower $n \rightarrow$ smoother transition

Butterworth Low-Pass Filter (BLPF)

Property

Transition band

Ringing

Smoothness

Flexibility

Complexity

Behaviour

Gradual (controlled by n)

Very small (much less than Ideal)

Good

Order n controls sharpness

Moderate

Application

Medical images

General denoising

Images requiring contrast preservation

Preprocessing for segmentation

Why

Smooth without extreme distortions

Removes noise without strong artifacts

Better edge behavior

Smooth output without ringing

Gaussian Low-Pass Filter (GLPF)

- A perfectly smooth filter — no ringing, extremely natural smoothing.

$$H(u, v) = e^{-\frac{D(u, v)^2}{2D_0^2}}$$

Gaussian Low-Pass Filter (GLPF)

Property

Transition

Ringing

Smoothness

Edge preservation

Complexity

Behaviour

Smoothest of all (no slope discontinuity)

None (best in this regard)

Excellent

Reasonably good

Slightly heavier but manageable

Gaussian Low-Pass Filter (GLPF)

Application

Natural image smoothing
Removing Gaussian noise
Computer vision preprocessing
Medical imaging

Why

Most visually pleasant
Matches noise distribution
Does not distort edges sharply
Important for reducing artifacts

Comparison Summary: Ideal vs Butterworth vs Gaussian

Property	Ideal LPF	Butterworth LPF	Gaussian LPF
Cutoff	Sharp	Smooth (controlled by n)	Very smooth
Transition	Abrupt	Moderate	Smoothest
Ringings	High	Low	None
Natural appearance	Poor	Good	Best
Complexity	Low	Medium	Medium
Best for	Synthetic images	General smoothing	Natural images

Comparison Summary: Ideal vs Butterworth vs Gaussian

Which filter should you use?

- For **teaching or demonstration** → **Ideal LPF**
- For **balanced smoothing** → **Butterworth (n=2 or 3)**
- For **natural, artifact-free output** → **Gaussian LPF**



The End